Compression de données avec l'algorithme de Huffman

1 Introduction

1.1 Récupération des fichiers

Depuis la ligne de commande, naviguez vers un répertoire vous appartenant, et exécutez la commande suivante, qui téléchargera et décompressera un répertoire nommé `huffman` contenant plusieurs fichiers dont une source OCaml :

```
curl cdn.sci-phy.org/mp2i/tp22-huffman.tgz | tar xvz
```

Le fichier à compléter est le fichier `huffman.ml`. On y trouvera quelques fonctions utiles qui seront présentées au fur et à mesure de nos besoins. On y trouve également trois fichiers de tests contenant du texte, `kanagawa.txt`, `adn.dat` et `declaration.txt`.

1.2 Objectifs

Dans ce sujet, on s'intéresse à la compression d'une chaîne de caractères (et à sa décompression) par la méthode de Huffman, qui encode chaque caractère par un code (succession de bits) choisi de façon à minimiser la longueur totale du message encodé.

2 Compression

2.1 Détermination des fréquences

Préalablement à la construction de l'arbre des codes de Huffman, il nous faut déterminer les fréquences des différents symboles dans le texte à compresser.

1. Proposer une fonction `char_counts` de signature `string -> int array` prenant en argument une chaîne de caractères et retournant un tableau de taille 256 tel que, pour tout $0 \leqslant i < 256$, le contenu de la case d'index i du tableau corresponde au nombre d'occurence du caractère de point de code i dans la chaîne. On rappelle que l'on peut obtenir le point de code d'un caractère `c` en écrivant « `int_of_char c` ». La fonction `char_of_int` réalise l'opération inverse.

2. Tester la fonction précédente sur une chaîne simple, par exemple « `"PARTERRE"` ».

2.2 Construction de l'arbre

La méthode de compression de Huffman consiste à construire ce que l'on appelle un arbre binaire balancé, dont les feuilles sont les caractères présents dans le texte à compresser, et construit de façon à ce que, pour tout sous-arbre, le nombre d'occurrence des caractères dans le sous-arbre gauche et le nombre d'occurrences des caractères dans le sous-arbre droit soit aussi proches que possible.

Pour ce faire, on procède de la façon suivante : on définit un type `tree` par :

```
type tree =
  | Leaf of int * int
  | Node of int * tree * tree
```

Pour les feuilles, le second entier est le point de code du caractère considéré (par exemple 65 pour `'A'`, on rappelle qu'on peut obtenir le point de code d'un caractère avec la fonction `int_of_char`), le premier correspond au nombre d'occurrences de ce caractère dans le texte à compresser. Pour les nœuds internes, outre les deux sous-arbres, l'entier correspond au nombre d'occurrence de l'*ensemble* des caractères dans ses feuilles.

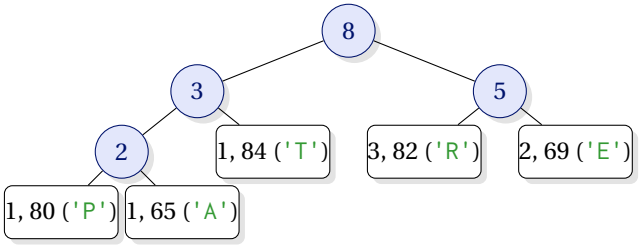
3. Proposer une fonction `nb_codes` de signature `tree -> int` prenant un arbre et retournant le nombre d'occurrences dans le texte des caractères qu'il contient.

4. Proposer une fonction `merge` de signature `tree -> tree -> tree` prenant en argument deux arbres et construisant un arbre dont les deux arbres en argument sont les sous-arbres, en respectant l'invariant précédemment décrit.

Pour construire l'arbre équilibré, on procède de la façon suivante :

- on crée une liste contenant autant d'arbres-feuilles que de caractères présents dans le texte à compresser ;
- on trie cette liste par fréquences croissantes (en cas d'égalité, l'ordre n'a pas d'importance) ;
- puis, tant que la liste contient au moins deux arbres :
  - on prend les deux premiers arbres de la liste et on les fusionne ;
  - on *insère* l'arbre résultat dans la liste, de façon à ce que la liste reste triée par nombre d'occurrences croissantes.

L'application de cet algorithme au mot PARTERRE conduit à obtenir un arbre de la sorte (il n'est pas unique, puisque certains caractères ont la même fréquence) :



Cet arbre servira à attribuer un code binaire unique à chaque caractère présent dans le texte. Pour ce faire, on considère la branche dont la feuille est un caractère donné. Chaque passage vers un fils gauche est associé à un 0, vers un fils droit à un 1. Ainsi, P est associé au code binaire 000, A au code 001, T au code 01, R au code 10. On remarque que les caractères plus fréquents seront associés à des codes plus courts (de façon similaire au code Morse, basé sur des idées similaires, même si le code Morse est un code ternaire : point, trait, silence).

Ainsi, le mot « PARTERRE » pourra être encodé par la séquence de bits

```
000 001 10 01 11 10 10 11
```

Ou plus précisément par

```
00001100111101011
```

puisqu'il n'est pas nécessaire de séparer les différents codes. On remarque qu'il n'est besoin que de 18 bits pour encoder les 8 caractères du mot!

Dans un premier temps, nous allons nous attacher à la construction de l'arbre, à partir de l'algorithme décrit précédemment. Pour ce faire, commençons avec quelques fonctions auxiliaires.

5. Proposer une fonction `insert` de signature `tree -> tree list -> tree list` qui effectue l'opération d'insertion.

6. En déduire une fonction `sort` de signature `tree list -> tree list` qui trie une liste d'arbres (par nombre d'occurrences croissant).

Note : plutôt que de trier la liste à chaque étape de l'algorithme de construction de l'arbre, on peut être tenté (à raison) d'utiliser un tas. Toutefois, le temps nécessaire à la construction de l'arbre, même avec un algorithme quadratique en son nombre de feuilles, sera rapidement petit devant le temps passé à encoder le texte lui-même, aussi travaillerons-nous ici de façon moins efficace mais plus simple afin de garder du temps pour le reste.

7. Proposer une fonction `build_tree` de signature `int array -> tree` prenant en argument un tableau de fréquences et renvoyant un arbre de Huffman correspondant à ces fréquences.

En fait, on va avoir besoin d'un code particulier, permettant de signifier que l'on est parvenu à la fin du texte, de façon similiaire au code `\0` lorsque l'on encode une chaîne en C. Mais on ne veut pas s'interdire de compresser des fichiers contenant un quelconque code, aussi va-t-on utiliser la valeur 256 (qui ne correspond pas à un point de code existant, lesquels sont entre 0 et 255). Ce code n'apparaîtra qu'une seule fois, à la fin du texte.

8. Modifier la fonction précédente pour inclure le code 256 (de nombre d'occurrence 1) dans l'arbre de Huffman.

9. Tester les fonctions `char_counts` et `build_tree` sur des chaînes simples ne contenant qu'un nombre limité de caractères, par exemple « PARTERRE », et vérifier que l'arbre obtenu est conforme aux attentes.

On fournit une fonction `read_file` prenant en argument un nom de fichier et retournant une chaîne de caractères correspondant à son contenu.

10. Lire le contenu du fichier « `adn.dat` », construire la table des occurrences et l'arbre correspondant. Vérifier qu'il est cohérent.

11. Faire de même avec les autres fichiers fournis (évidemment, il n'est pas question d'analyser les arbres ainsi construits en détail).

2.3 Construction du dictionnaire

Utiliser l'arbre pour encoder un texte est malcommode. On préfère donc disposer d'un dictionnaire associant, à chaque code entier entre 0 et 256 (inclus) correspondant à un point de code *présent* dans l'arbre ou au code spécial de fin, une liste de booléens correspondant à la séquence de bits permettant de les coder.

12. Écrire une fonction `tree_to_dict` de signature `tree -> (int, bool list) Hashtbl.t` permettant de construire le dictionnaire correspondant. On cherchera une solution dont la complexité est la taille de l'arbre (en supposant les opérations sur le dictionnaire de coût unitaire).

2.4 Tests préliminaires

13. Écrire une fonction `ratio` de signature prenant en argument une chaîne de caractères et un dictionnaire construit à partir d'un arbre de Huffman et retournant un flottant représentant le taux de compression (nombre de bits du texte compressé sur nombre de bits du texte original) obtenu en encodant le texte avec l'arbre (on considérera uniquement, pour le moment, le nombre de bits nécessaire à l'encodage du texte proprement dit, en ignorant le besoin d'y adjoindre l'arbre). Les caractères du fichier original sont encodés sur 8 bits. Si la taille du texte compressé est inférieure à celle du texte original, ce ratio sera inférieur à 1 (plus le ratio est petit, plus la compresson est efficace).

14. En utilisant les fonctions écrites, mesurer les taux de compression obtenus sur les fichiers proposés.

15. Faire de même sur une chaîne de 10000 caractères dont les points de code sont choisis aléatoirement entre 1 et 255 avec une loi uniforme (on évitera le point de code 0 qui peut occasionner des soucis avec certains IDE). On pourra par exemple générer la chaîne de la façon suivante :

```
String.init 10000 (fun i -> char_of_int (1+Random.int 255))
```

3 Écriture du fichier compressé

3.1 Fonctions disponibles

Pour écrire bit à bit dans un fichier, on fournit trois fonctions :

- une fonction `open_bit_out` prenant en argument un nom de fichier, ouvrant un fichier sous ce nom en écriture, et retournant un descripteur (un objet de type `outbitfile`) permettant de travailler avec le fichier ouvert ;
- une fonction `output_bit` prenant en argument un descripteur et un booléen et ne retournant rien, mais ajoutant un bit au contenu du fichier ;
- une fonction `close_bit_out` prenant en argument un descripteur, fermant le fichier (et achevant les écritures de bits n'ayant pas encore été effectuées).

Pour éviter les soucis liés au fait que l'on ne dispose en théorie que de fonctions travaillant sur des fichiers texte dans le programme, la méthode utilisée n'**encode que six bits par caractère présent dans le fichier de sortie et non huit** : elle n'utilise que les points de code 32 à 95. Le fichier produit est donc sans risque parfaitement lisible (mais plus long de 33% de ce qu'il pourrait être avec une sortie réellement en binaire).

3.2 Éncodage du texte

On se préoccupe dans un premier temps de l'écriture du texte proprement dit encodé par l'arbre.

16. Proposer une fonction `encode_char` de signature `outbitfile -> int * bool list Hashtbl.t -> int -> unit` prenant en argument un descripteur, un dictionnaire contenant les codes d'un arbre de Huffman et un entier entre 0 et 256 présent parmi les clés de l'arbre, et enregistrant dans le fichier les bits correspondants à cet entier.

17. En déduire une fonction `encode_text` de signature `outbitfile -> int * bool list Hashtbl.t -> string -> unit` effectuant la même chose pour une chaîne de caractères. **On terminera en ajoutant, à la toute fin du texte, le code correspondant à l'entier** 256 **qui signifiera la fin du fichier** (cela nous sera utile lors du décodage).

3.3 Encodage de l'arbre

Pour pouvoir décoder le fichier, il est indispensable d'enregistrer également l'arbre dans le fichier de sortie. Pour ce faire, nous allons sérialiser (transformer en une séquence de bits) l'arbre de la façon suivante : on explore l'arbre selon un parcours en profondeur suffixe ; chaque feuille est encodée comme une séquence de dix bits : un zéro, suivi des neufs bits correspondant à la représentation binaire de l'entier entre 0 et 256 correspondant à son étiquette ; un nœud interne est encodé comme un unique bit à 1.

18. Proposer une fonction `encode_leaf` de signature `outbitfile -> int -> unit` prenant en argument un descripteur et un entier entre 0 et 256 et encodant dans le fichier une feuille correspondant à ce point de code.

19. En déduire une fonction `encode_tree` de signature `outbitfile -> tree -> unit` encodant l'arbre de Huffman passé en paramètre dans le fichier.

3.4 Constitution du fichier compressé

Pour permettre la décompression, le fichier compressé contiendra les éléments suivants :

- l'arbre, correctement encodé ;
- **un unique bit à** 1 **qui servira de séparation** ;
- le texte, encodé avec les codes de Huffman ;
- le code de Huffman correspondant au code spécial 256 ;
- potentiellement quelques bits supplémentaires de « padding ».

20. En déduire une fontion `compress` de signature `string -> string -> unit` prenant en argument une chaîne de caractères à compresser, une chaîne de caractères contenant un nom de fichier, et enregistrant la première chaîne selon l'algorithme de Huffman dans un fichier portant un nom correspondant à la seconde chaîne.

21. Comparer la taille des fichiers obtenus à la taille originale des fichiers (on rappelle que pour des raisons pratiques, les fichiers obtenus sont 33% plus gros qu'ils ne l'auraient été avec l'algorithme authentique).

4 Décompression

4.1 Fonctions disponibles

Pour lire bit à bit dans un fichier, on fournit trois fonctions :

- une fonction `open_bit_in` prenant en argument un nom de fichier, ouvrant un fichier sous ce nom en lecture, et retournant un descripteur (un objet de type `inbitfile`) permettant de travailler avec le fichier ouvert ;
- une fonction `read_bit` prenant en argument un descripteur, et lisant puis retournant un booléen correspondant à un bit lu dans le fichier ;
- une fonction `close_bit_in` prenant en argument un descripteur, fermant le fichier.

Comme le fichier ne peut contenir qu'un nombre de bits divisible par six, et que l'information du nombre de bits n'a pas été enregistrée, il peut arriver que l'on puisse lire quelques bits supplémentaires qui n'ont pas été explicitement écrits. Il n'y a pas non plus de précautions évitant de lire au-delà de la fin du fichier, les fonctions utilisant le mécanisme de lecture doivent savoir quand s'arrêter !

4.2 Reconstitution de l'arbre

La première chose à faire, pour reconstituer le texte original, est de reconstruire l'arbre de Huffman. Pour ce faire, on procède de la façon suivante : on crée une pile d'arbres vide

(on représentera la pile comme une référence vers une liste), puis, itérativement

- on lit un bit dans le fichier
- s'il s'agit d'un 0, on lit 9 bits supplémentaires dans le fichier et on crée une feuille (on pourra garder le nombre d'occurrence à 0, il n'a plus d'importance) que l'on place dans la pile
- s'il s'agit d'un 1, on extrait deux arbres de la pile, on construit un nouvel arbre dont les sous-arbre gauche et droit correspondent aux deux sous-arbres extrait (dans quel ordre faut-il les placer ?) et replace le nouvel arbre dans la pile

Le processus se déroule tant qu'il est possible d'extraire deux arbres de la pile lorsque l'on obtient un 1 : lorsqu'un 1 est extrait du fichier et que la pile ne contient qu'un arbre, alors il s'agit de l'arbre entier (le 1 extrait du fichier est celui qui sépare l'encodage de l'arbre de l'encodage du texte, il ne fait pas partie du parcours suffixe !)

22. Proposer une fonction `decode_tree` de signature `inbitfile -> tree` extrayant un arbre d'un fichier préalablement ouvert.

23. Vérifier que l'arbre extrait est bien le bon (on pourra vérifier en priorité le fichier ADN dont l'arbre est simple).

4.3 Reconstitution du texte

24. Proposer une fonction `decode_text` de signature `inbitfile -> tree -> string` qui, à partir d'un fichier (dont on a lu le début correspondant à son arbre) et un arbre de Huffman, reconstitue la chaîne originale. On arrêtera le décodage lorsque l'on lira le code spécial 256.

25. En déduire une fonction `deflate` de signature `string -> string` prenant en argument une chaîne de caractère correspondant à un nom de fichier contenant un texte compressé par l'algorithme de Huffman et retournant une chaîne de caractères correspondant à son contenu.

26. Vérifier que l'on parvient bien à récupérer le contenu des fichiers originaux.